

Testing of a Symmetric Encryption Algorithm: ChaCha20 Performance Analysis

Name: Irmak Yılmaz

Student Number: E-379

1. Algorithm Selection

Selected Algorithm: ChaCha20 (A modern, fast stream cipher often used in lightweight and edge computing contexts).

Reason for Selection: As a Data Science and AI enthusiast, I know that Machine Learning models require massive amounts of data. This data is often collected from IoT sensors, mobile phones, and edge devices. These devices have limited CPU and battery. ChaCha20 is incredibly fast in software and consumes very little RAM, making it perfect for encrypting large datasets on edge devices before sending them to a central database for AI training.

How it Works: ChaCha20 is a symmetric stream cipher. Unlike block ciphers (like AES) that encrypt data in fixed chunks, ChaCha20 encrypts data bit-by-bit or byte-by-byte. It uses a 256-bit key and a nonce (a unique number) to generate a random stream of data, which is then XOR'ed with our original dataset to create the ciphertext.

2. Preparation of the Testing Environment

Tools Used: Since I need to test large datasets (up to 100 MB) and measure CPU/RAM usage accurately, online tools are not sufficient. Therefore, I wrote a custom testing script using the **Python** programming language. Python is the industry standard for Data Science, and it allows me to use libraries like psutil (for CPU/RAM monitoring) and cryptography (for the ChaCha20 implementation).

Input Data: To simulate real-world datasets, my Python script automatically generates three dummy text files with exact sizes:

- dataset_1MB.txt (1 Megabyte)
- dataset_10MB.txt (10 Megabytes)
- dataset_100MB.txt (100 Megabytes)

3. Conducting the Tests

I used a custom Python script to encrypt and decrypt the generated datasets (1 MB, 10 MB, and 100 MB) using the ChaCha20 algorithm with a 256-bit key. The script successfully verified that the decrypted data perfectly matched the original plaintext for all file sizes.

Here are the performance results obtained from my local testing environment:

- **1 MB Dataset:**
 - Encryption: 0.0047 seconds | CPU Usage: 0.0% | RAM Consumption: 2.03 MB

- Decryption: 0.0020 seconds | CPU Usage: 0.0% | RAM Consumption: 1.01 MB
- **10 MB Dataset:**
 - Encryption: 0.0076 seconds | CPU Usage: 0.0% | RAM Consumption: 10.00 MB
 - Decryption: 0.0066 seconds | CPU Usage: 33.3% | RAM Consumption: 10.00 MB
- **100 MB Dataset:**
 - Encryption: 0.0624 seconds | CPU Usage: 24.7% | RAM Consumption: 100.00 MB
 - Decryption: 0.0662 seconds | CPU Usage: 24.3% | RAM Consumption: 100.00 MB

```
PROBLEMS 1 OUTPUT DEBUG CONSOLE TERMINAL PORTS
PS C:\Users\23022\OneDrive\Desktop\ders> python crypto_test.py
=== Starting ChaCha20 Performance Test ===

Creating [dataset_1MB.txt] (1 MB)...

--- Testing 1 MB Dataset ---
ENCRYPTION -> Time: 0.0047 sec | CPU: 0.0% | RAM spike: 2.03 MB
DECRYPTION -> Time: 0.0020 sec | CPU: 0.0% | RAM spike: 1.01 MB
Status: Success! Decrypted data matches original plaintext.
Creating [dataset_10MB.txt] (10 MB)...

--- Testing 10 MB Dataset ---
ENCRYPTION -> Time: 0.0076 sec | CPU: 0.0% | RAM spike: 10.00 MB
DECRYPTION -> Time: 0.0066 sec | CPU: 33.3% | RAM spike: 10.00 MB
Status: Success! Decrypted data matches original plaintext.
Creating [dataset_100MB.txt] (100 MB)...

--- Testing 100 MB Dataset ---
ENCRYPTION -> Time: 0.0624 sec | CPU: 24.7% | RAM spike: 100.00 MB
DECRYPTION -> Time: 0.0662 sec | CPU: 24.3% | RAM spike: 100.00 MB
Status: Success! Decrypted data matches original plaintext.
```

4. Analysis of Results

Impact of Data Size on Execution Time: The test results show a direct, linear relationship between the data size and the execution time. Encrypting a large 100 MB dataset took only 0.0624 seconds. This proves that ChaCha20 is extremely fast and highly optimized for software environments. Furthermore, the encryption and decryption times are almost identical, demonstrating consistent and balanced performance.

Impact on Resource Consumption (CPU and RAM): The RAM consumption increases proportionally with the file size. For example, processing the 100 MB file required exactly 100.00 MB of RAM. This is normal and expected because the Python script loads the entire file into memory during the process. The CPU usage remained remarkably efficient. For smaller files, it was mostly at 0%, and for the massive 100 MB file, it stabilized at around 24.5%. The brief 33.3% CPU spike during the 10 MB decryption is just a normal, temporary system operation to initiate the task.

Conclusion: The ChaCha20 algorithm demonstrated outstanding performance. In modern Data Science and Artificial Intelligence projects, developers constantly need to securely transfer large datasets from small edge devices (such as IoT sensors or mobile applications) to backend servers (like a FastAPI backend handling data for machine learning models). This experiment proves that ChaCha20 provides high-level security with its 256-bit key without draining the battery, consuming excessive memory, or overloading the CPU of lightweight devices.

Note on Key Lengths: The standard implementation of the ChaCha20 algorithm uses a fixed 256-bit key to guarantee a very high level of security. Therefore, testing and comparing multiple key lengths (such as 128-bit or 192-bit) was not applicable for this specific algorithm.